

DAY 18

Real Client Blockchain Project From Brief to Production-Ready Handoff

60-minute beginner lecture Simulate a real outsourcing engagement: requirement discovery, scope, architecture, Solidity implementation, tests, frontend/backend integration, testnet release, verification, documentation and client handoff.

Career focus: Learn how to turn blockchain knowledge into a professional delivery process that a LinkedIn or Upwork client can understand, review, test and pay for.

Case study: A client wants an ERC-20 staking dApp where users approve tokens, stake, earn time-based rewards, claim, withdraw, see their position in a frontend, and receive a clear deployment/handoff package.

Technical basis: This lecture uses modern Foundry, OpenZeppelin Contracts 5.x and ethers.js v6 patterns. Production dependencies should always be rechecked before a live deployment.

1. Learning Objectives & 60-Minute Teaching Plan

By the end of Day 18, you should be able to take a short client message and convert it into a scoped blockchain project with explicit requirements, security assumptions, milestones, acceptance tests and delivery evidence.

Time	Topic	Outcome
0-5 min	Read the client brief	Separate business goal from implementation guesses.
5-12 min	Discovery + scope	Ask the questions that control price, risk and architecture.
12-20 min	Requirements + acceptance criteria	Convert vague statements into testable deliverables.
20-28 min	Architecture + contract design	Choose components, state, roles, events and trust boundaries.
28-38 min	Implementation walkthrough	Build a simple client-ready staking core.
38-46 min	Testing + security	Define unit, fuzz, invariant and integration coverage.
46-52 min	Frontend/backend + release	Wire the dApp, events, deployment config and testnet QA.
52-57 min	Handoff + client communication	Package evidence, docs, addresses, roles and support notes.
57-60 min	Scenario review	Debug one client issue and summarize the professional workflow.

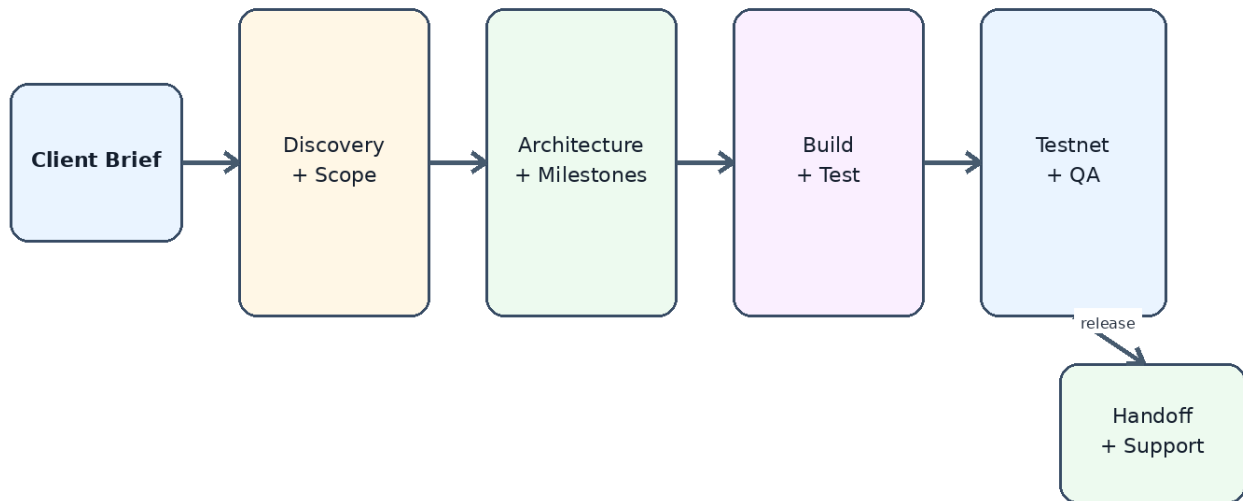
60-second target explanation: A professional blockchain job is not code alone. It is a traceable chain from client need to requirement, implementation, test evidence, deployment evidence and handoff documentation. If a requirement cannot be tested, it is not yet precise enough.

2. The Simulated Client Brief

Client message We need a staking dApp for our ERC-20 token. Users should connect a wallet, stake tokens, earn rewards, claim rewards and withdraw. We want a clean frontend and deployment on an EVM testnet first, then mainnet after testing.

A beginner may immediately start writing `stake()` and `withdraw()`. A freelancer should first identify the missing information because those missing choices control contract economics, security and delivery scope.

Real Client Delivery Lifecycle



2.1 Hidden questions inside one sentence

- Which exact token contract and network?
- Is the staking token also the reward token?
- Where do rewards come from and who funds them?
- Fixed APR, fixed reward rate, campaign budget or admin-configurable emission?
- Can rewards run out? What should happen then?
- Is there a lock period, penalty, minimum stake or maximum stake?
- Can the admin pause staking? Can the admin recover unrelated tokens?
- Is the contract upgradeable or immutable?
- Who owns/administers production contracts - one wallet or multisig?
- Which frontend framework, wallet support, backend/indexer and hosting are in scope?

3. Discovery: Ask Before You Estimate

Area	Questions to ask the client
Business	What behavior must users experience? What is the launch goal and target date?
Token	Address, decimals, supply model, transfer restrictions, fee-on-transfer behavior?
Rewards	Formula, funding source, duration, rate updates, unclaimed rewards, insolvency behavior?
Roles	Who can fund, pause, change rate, recover tokens or upgrade?
Network	Target chain, testnet, gas expectations, explorer, RPC provider?
Frontend	Existing repo or new app? Wallets? Responsive design? Branding?
Backend	Need event indexer, API, notifications, analytics or admin dashboard?
Security	Audit required? Multisig? Timelock? Upgradeability? Emergency pause?
Delivery	Source repo, tests, deployment scripts, verified contracts, docs, handoff call?

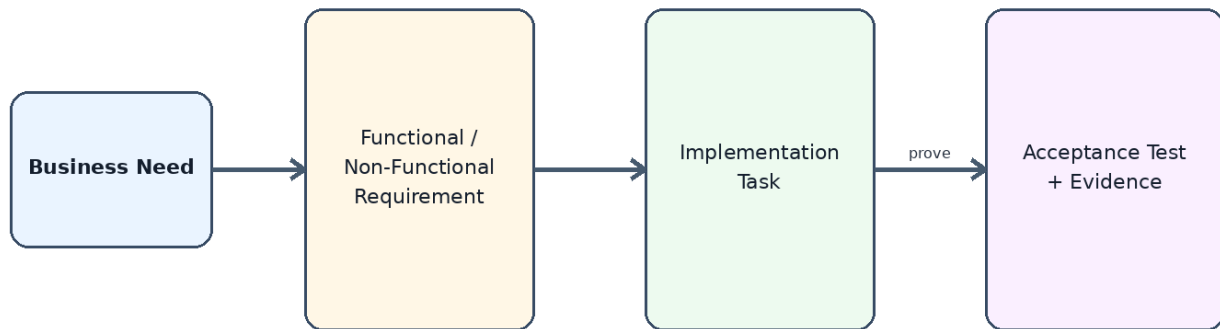
Freelancer habit: Write the client's answers into the project specification. Do not rely on chat memory for economic rules or production permissions.

4. Convert the Brief into Requirements

Requirements should be explicit enough that you can write a test for them. Separate functional behavior from non-functional expectations.

ID	Type	Requirement	Proof
FR-01	Functional	User can stake an approved amount greater than zero.	Unit test + testnet transaction
FR-02	Functional	User can withdraw up to their staked balance.	Unit/fuzz tests
FR-03	Functional	Rewards accrue according to agreed formula.	Time-warp unit tests
FR-04	Functional	User can claim available rewards.	Balance/event assertions
FR-05	Functional	UI displays wallet, staked balance and earned rewards.	Acceptance test screenshots
NFR-01	Security	Admin functions require explicit role authorization.	Negative access-control tests
NFR-02	Security	Stake/withdraw/claim resist reentrancy.	Design review + tests
NFR-03	Quality	Repository includes deterministic build/test commands.	README + CI run
NFR-04	Delivery	Contracts verified and deployment metadata recorded.	Explorer links + release file

Requirement Traceability Flow



5. Scope, Assumptions and Out-of-Scope Items

Scope protects both freelancer and client. It prevents a small staking job from silently becoming a token redesign, audit, bridge, mobile app and production DevOps project.

In scope for this Day 18 project	Explicitly out of scope unless added
Staking smart contract	Changing the client token contract
Reward accounting	Tokenomics consulting / APY design
Foundry unit/fuzz tests	Independent third-party security audit
React/ethers integration example	Full custom design system
Basic event monitoring	Enterprise analytics warehouse
Testnet deployment + verification	Mainnet launch approval/signing
Deployment/handoff documentation	Legal/regulatory review
One target EVM chain	Cross-chain bridge or Solana port

5.1 Assumption log

ASSUMPTIONS.md

- A1. Staking token implements standard ERC-20 behavior.
- A2. Reward token is standard ERC-20 and pre-funded by admin.
- A3. Rewards use a fixed rewardsDuration campaign model.
- A4. No lock period or early-withdraw penalty in v1.
- A5. Production admin will be a client-controlled multisig.
- A6. Mainnet deployment is a separate release gate.

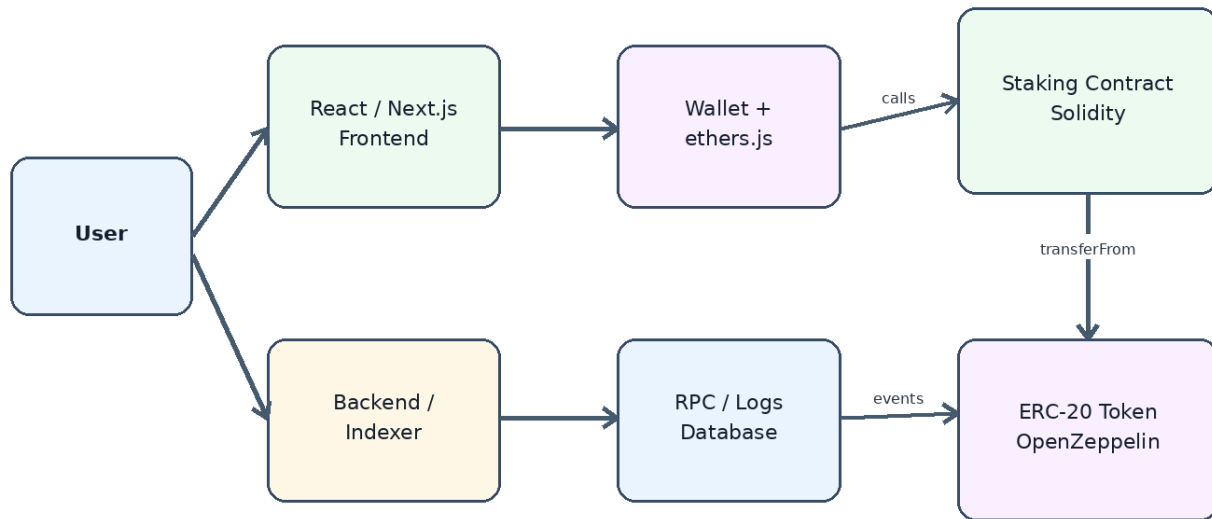
6. Acceptance Criteria: Define "Done" Before Coding

Feature	Acceptance criteria
Stake	Approved user stakes 100 tokens; staked balance increases by exactly 100; Staked event emitted.
Zero stake	stake(0) reverts with a documented custom error.
Withdraw	User cannot withdraw more than their recorded stake.
Rewards	After configured elapsed time, earned(user) matches the specified accounting model within integer rounding rules.
Claim	Claim transfers only the caller's accrued reward and resets stored reward state correctly.
Pause	When paused, stake/withdraw/claim behavior follows the written pause policy.
Frontend	Wrong network is detected; writes require wallet signer; pending/confirmed/error UI states are visible.
Deployment	Source verified; address, chain ID, tx hash, compiler and commit recorded.
Handoff	Client receives repo, README, env template, test report, deployment record and admin-role checklist.

Powerful rule: If the client and developer cannot agree how to prove a requirement is complete, the requirement is not ready for implementation.

7. Architecture Before Code

Client Project Architecture - Staking dApp



Component	Responsibility
Staking contract	Custody stake, account rewards, enforce roles, emit events.
ERC-20 token	Asset users stake; may be a separate reward token.
Frontend	Wallet connection, reads, writes, transaction UX.
Wallet	User signing; never controlled by frontend code.
RPC provider	Blockchain reads, transaction broadcast and event access.
Backend/indexer	Optional historical event indexing, analytics and notifications.
Database	Off-chain query cache only; not the source of truth for on-chain balances.

Architecture boundary: The blockchain contract is authoritative for stake/reward state. A database may index it for speed, but the database must not silently become the source of truth for funds.

8. Data Model and Contract State

Before implementation, write the state variables and invariants in plain language.

State	Meaning	Invariant / risk
totalSupply	Total tokens recorded as staked	Should equal sum of user balances in the model.
balanceOf[user]	Stake credited to a user	Cannot be negative; withdraw cannot exceed it.
rewardRate	Rewards emitted per second	Must be economically funded.
rewardPerTokenStored	Global cumulative reward accumulator	Monotonic while rewards accrue.
userRewardPerTokenPaid[user]	User checkpoint	Updated whenever user stake/reward changes.
rewards[user]	Stored accrued reward	Claim should transfer and reset atomically.
periodFinish	Campaign end timestamp	Accrual stops after finish.

8.1 Core accounting property

```

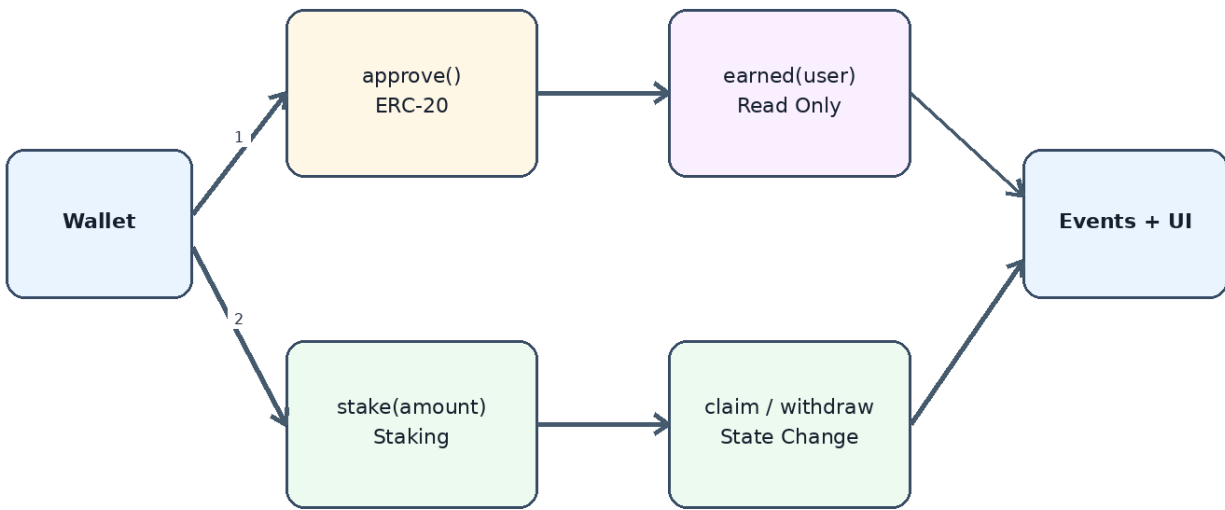
earned(user) =
  balanceOf[user] *
  (rewardPerToken() - userRewardPerTokenPaid[user]) / 1e18
+ rewards[user]

```

Client clarification: The formula above is an implementation model, not a promise of APR. A business statement such as "20% APY" must first be translated into an exact funding/rate policy.

9. User Flow: Approval, Stake, Earn, Claim, Withdraw

Staking User Flow



1. Frontend reads token allowance for the staking contract.
2. If allowance is insufficient, user sends ERC-20 approve().
3. User sends stake(amount).
4. Contract updates reward checkpoints before changing stake balance.
5. Contract pulls tokens with SafeERC20.safeTransferFrom().
6. Frontend displays staked balance and earned(user).
7. User may claim reward or withdraw stake according to policy.
8. Events feed UI refresh and optional backend indexing.

9.1 What belongs on-chain?

On-chain	Off-chain
Stake balances	Chart-friendly historical aggregates
Reward accounting checkpoints	Email/Discord notifications
Admin authorization	Search/filter views
Pause state	Cached token metadata
Events proving actions	UI preferences

10. Contract Skeleton: Imports, State and Roles

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";

contract ClientStaking is AccessControl, ReentrancyGuard, Pausable {
    using SafeERC20 for IERC20;

    bytes32 public constant REWARD_MANAGER_ROLE = keccak256("REWARD_MANAGER_ROLE");
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");

    IERC20 public immutable stakingToken;
    IERC20 public immutable rewardsToken;

    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;

    uint256 public rewardRate;
    uint256 public rewardsDuration = 7 days;
    uint256 public periodFinish;
    uint256 public lastUpdateTime;
    uint256 public rewardPerTokenStored;
    mapping(address => uint256) public userRewardPerTokenPaid;
    mapping(address => uint256) public rewards;

    event Staked(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);
    event RewardPaid(address indexed user, uint256 reward);
    event RewardAdded(uint256 reward);

    error ZeroAmount();
    error InsufficientStake();
}
```

11. Contract Skeleton: Reward Accounting + Modifier

```

constructor(IERC20 _stakingToken, IERC20 _rewardsToken, address admin) {
    stakingToken = _stakingToken;
    rewardsToken = _rewardsToken;
    _grantRole(DEFAULT_ADMIN_ROLE, admin);
    _grantRole(REWARD_MANAGER_ROLE, admin);
    _grantRole(PAUSER_ROLE, admin);
}

modifier updateReward(address account) {
    rewardPerTokenStored = rewardPerToken();
    lastUpdateTime = lastTimeRewardApplicable();
    if (account != address(0)) {
        rewards[account] = earned(account);
        userRewardPerTokenPaid[account] = rewardPerTokenStored;
    }
    _;
}

function lastTimeRewardApplicable() public view returns (uint256) {
    return block.timestamp < periodFinish ? block.timestamp : periodFinish;
}

function rewardPerToken() public view returns (uint256) {
    if (totalSupply == 0) return rewardPerTokenStored;
    return rewardPerTokenStored +
        ((lastTimeRewardApplicable() - lastUpdateTime) * rewardRate * 1e18)
        / totalSupply;
}

function earned(address account) public view returns (uint256) {
    return (balanceOf[account] *
        (rewardPerToken() - userRewardPerTokenPaid[account])) / 1e18
        + rewards[account];
}

```

Review question: Why update rewards before changing balance? Because reward accounting must checkpoint the user and global accumulator using the old stake amount before applying the new state.

12. Contract Skeleton: Stake, Withdraw and Claim

```
function stake(uint256 amount)
    external nonReentrant whenNotPaused updateReward(msg.sender)
{
    if (amount == 0) revert ZeroAmount();
    totalSupply += amount;
    balanceOf[msg.sender] += amount;
    stakingToken.safeTransferFrom(msg.sender, address(this), amount);
    emit Staked(msg.sender, amount);
}

function withdraw(uint256 amount)
    public nonReentrant updateReward(msg.sender)
{
    if (amount == 0) revert ZeroAmount();
    if (amount > balanceOf[msg.sender]) revert InsufficientStake();
    totalSupply -= amount;
    balanceOf[msg.sender] -= amount;
    stakingToken.safeTransfer(msg.sender, amount);
    emit Withdrawn(msg.sender, amount);
}

function getReward() public nonReentrant updateReward(msg.sender) {
    uint256 reward = rewards[msg.sender];
    if (reward == 0) return;
    rewards[msg.sender] = 0;
    rewardsToken.safeTransfer(msg.sender, reward);
    emit RewardPaid(msg.sender, reward);
}

function exit() external {
    uint256 staked = balanceOf[msg.sender];
    if (staked != 0) withdraw(staked);
    getReward();
}
```

13. Contract Skeleton: Funding, Pause and Solvency Checks

```
function notifyRewardAmount(uint256 reward)
    external onlyRole(REWARD_MANAGER_ROLE) updateReward(address(0))
{
    // Reward tokens should be funded before or atomically with configuration.
    if (block.timestamp >= periodFinish) {
        rewardRate = reward / rewardsDuration;
    } else {
        uint256 remaining = periodFinish - block.timestamp;
        uint256 leftover = remaining * rewardRate;
        rewardRate = (reward + leftover) / rewardsDuration;
    }

    uint256 balance = rewardsToken.balanceOf(address(this));
    uint256 reservedStake = address(stakingToken) == address(rewardsToken)
        ? totalSupply : 0;
    require(balance >= reservedStake + rewardRate * rewardsDuration,
        "insufficient reward funding");

    lastUpdateTime = block.timestamp;
    periodFinish = block.timestamp + rewardsDuration;
    emit RewardAdded(reward);
}

function pause() external onlyRole(PAUSER_ROLE) { _pause(); }
function unpause() external onlyRole(PAUSER_ROLE) { _unpause(); }
}
```

Important limitation: This is a teaching contract for the project workflow, not a substitute for a full audit. A real production contract may need stronger reward-funding accounting, role separation, emergency recovery design and token-behavior constraints.

14. Security Design Review Before Tests

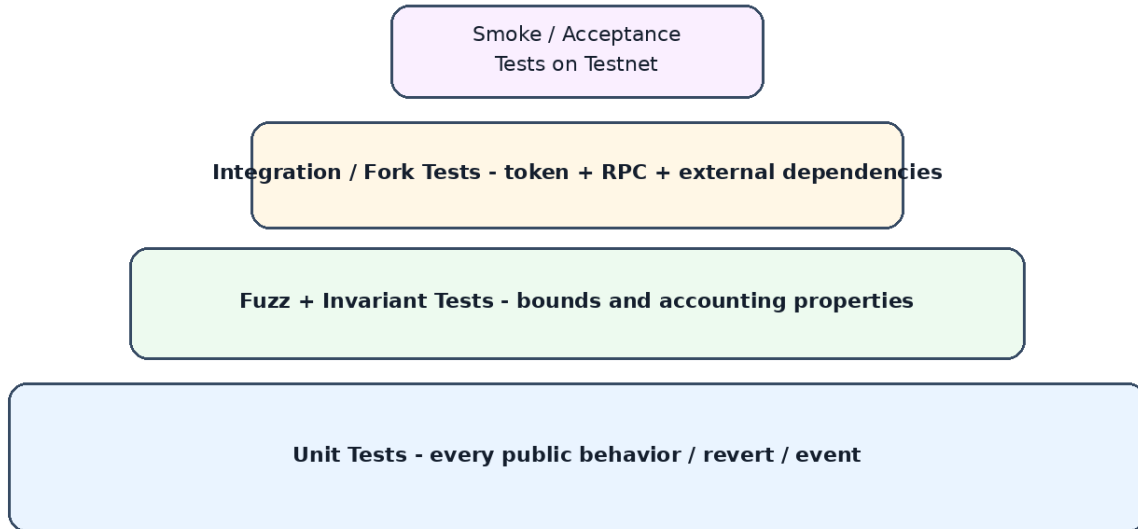
Threat / failure	Design response
Reentrancy during token transfer	Use CEI/accounting updates and ReentrancyGuard around write paths.
Unsafe ERC-20 return behavior	Use SafeERC20.
Unauthorized reward changes	Role-based access control; client-controlled admin.
Single-key admin compromise	Use multisig for production administration.
Reward insolvency	Validate funding policy and same-token reserved stake.
Fee-on-transfer/rebasing token	Either reject unsupported tokens or design accounting explicitly.
Paused funds trapped	Define whether withdraw remains available during pause.
Admin rug/recovery power	Document every privileged function and limit it.
Upgrade risk	Prefer immutable v1 unless upgrade requirement is explicit; otherwise use audited upgrade pattern.
Front-end wrong contract/network	Chain ID and deployed-address validation.

14.1 Security questions for the client

- Who receives DEFAULT_ADMIN_ROLE after launch?
- Should pause block stake only, or also claim/withdraw?
- Can any admin change rewards mid-campaign?
- What is the emergency plan if reward funding is wrong?
- Is a third-party audit a launch requirement?

15. Testing Strategy: Evidence, Not Hope

Client-Ready Test Layers



Layer	Examples
Unit	stake, withdraw, claim, reward update, pause, role checks, events, zero amount.
Revert / negative	withdraw too much, unauthorized notifyRewardAmount, paused stake, insufficient funding.
Fuzz	Random stake/withdraw amounts inside balances; reward amounts and elapsed time.
Invariant	totalSupply equals modeled sum of stake; user cannot withdraw more than credited; accounting never creates stake.
Integration	Real ERC-20 behavior, deployed bytecode, frontend ABI calls.
Fork	Optional external protocol/token behavior if the project depends on existing mainnet contracts.
Testnet acceptance	Wallet flow, approvals, pending UX, explorer verification, event indexing.

16. Foundry Unit Tests - Arrange, Act, Assert

```
contract ClientStakingTest is Test {
    ClientStaking staking;
    MockERC20 stakeToken;
    MockERC20 rewardToken;
    address alice = address(0xA11CE);
    address admin = address(this);

    function setUp() public {
        stakeToken = new MockERC20("Stake", "STK");
        rewardToken = new MockERC20("Reward", "RWD");
        staking = new ClientStaking(stakeToken, rewardToken, admin);

        stakeToken.mint(alice, 1_000 ether);
        vm.prank(alice);
        stakeToken.approve(address(staking), type(uint256).max);
    }

    function testStakeUpdatesState() public {
        vm.prank(alice);
        staking.stake(100 ether);
        assertEquals(staking.balanceOf(alice), 100 ether);
        assertEquals(staking.totalSupply(), 100 ether);
        assertEquals(stakeToken.balanceOf(address(staking)), 100 ether);
    }

    function testCannotWithdrawMoreThanStake() public {
        vm.startPrank(alice);
        staking.stake(100 ether);
        vm.expectRevert(ClientStaking.InsufficientStake.selector);
        staking.withdraw(101 ether);
        vm.stopPrank();
    }
}
```

17. Fuzz, Invariant and Time-Based Tests

```
function testFuzzStake(uint96 rawAmount) public {
    uint256 amount = bound(uint256(rawAmount), 1, 1_000 ether);
    vm.prank(alice);
    staking.stake(amount);
    assertEq(staking.balanceOf(alice), amount);
}

function testRewardAccruesOverTime() public {
    rewardToken.mint(address(staking), 700 ether);
    staking.notifyRewardAmount(700 ether);

    vm.prank(alice);
    staking.stake(100 ether);

    vm.warp(block.timestamp + 1 days);
    assertGt(staking.earned(alice), 0);
}
```

17.1 Invariants to document

- Recorded totalSupply never exceeds actual staking-token balance for standard tokens.
- A user cannot withdraw more principal than balanceOf[user].
- Claiming rewards cannot decrease user principal.
- Only authorized roles can change reward schedule or pause state.
- Reward checkpoint values should not move backward under normal time progression.

Testing mindset: Do not write tests only for the happy path shown in the client demo. Write tests for the exact ways money, permissions and accounting can fail.

18. Frontend Integration: Reads, Writes and UX States

The frontend is a transaction coordinator and presentation layer. It should not reimplement reward accounting as an authoritative balance.

```
import { BrowserProvider, Contract, parseUnits } from "ethers";

const provider = new BrowserProvider(window.ethereum);
await provider.send("eth_requestAccounts", []);
const signer = await provider.getSigner();

const staking = new Contract(STAKING_ADDRESS, STAKING_ABI, signer);
const token = new Contract(TOKEN_ADDRESS, ERC20_ABI, signer);

const amount = parseUnits("100", 18);
const approveTx = await token.approve(STAKING_ADDRESS, amount);
await approveTx.wait();

const stakeTx = await staking.stake(amount);
await stakeTx.wait();
```

UI state	User message
Disconnected	Connect wallet
Wrong chain	Switch to supported network
Approval required	Approve staking token
Wallet prompt	Confirm transaction in wallet
Submitted	Transaction submitted - show hash
Pending	Waiting for confirmation
Confirmed	Stake confirmed - refresh reads
Rejected	User rejected request
Reverted	Show decoded error / recovery guidance

19. Backend / Indexer: Optional but Valuable

A backend is useful for history, analytics and notifications, but it should derive blockchain facts from logs/reads and handle confirmation/reorg policy.

```
staking.on("Staked", async (user, amount, event) => {
  const log = event.log;
  await db.stakingEvents.upsert({
    txHash: log.transactionHash,
    logIndex: log.index,
    blockNumber: log.blockNumber,
    user,
    amount: amount.toString(),
    type: "STAKED"
  });
});
```

Backend concern	Professional behavior
Duplicates	Use txHash + logIndex or another deterministic event key.
Missed websocket event	Backfill from a stored block cursor.
Reorg	Wait chosen confirmations and reconcile removed/replaced logs.
RPC outage	Retry with backoff and provider failover policy.
Secrets	Server secrets only; never ship private keys to browser.
Database drift	Rebuildable from chain history where practical.

20. Repository Structure: Make the Project Easy to Handoff

```
client-staking/  
  contracts/  
    src/ClientStaking.sol  
    test/ClientStaking.t.sol  
    script/Deploy.s.sol  
    foundry.toml  
  frontend/  
    src/  
    package.json  
    .env.example  
  backend/  
    src/indexer.ts  
    package.json  
    .env.example  
  deployments/  
    sepolia.json  
    mainnet.example.json  
  docs/  
    REQUIREMENTS.md  
    ARCHITECTURE.md  
    SECURITY.md  
    TEST_PLAN.md  
    HANDOFF.md  
  README.md  
  .gitignore  
  .github/workflows/ci.yml
```

20.1 README minimum

- Project purpose and architecture
- Exact setup commands
- Supported Node/Foundry environment
- Environment-variable template
- Test commands
- Deployment commands
- Contract addresses by chain
- Known limitations
- Admin/role handoff notes

21. Deployment Configuration and Reproducibility

A client should be able to prove which source revision produced the deployed contract.

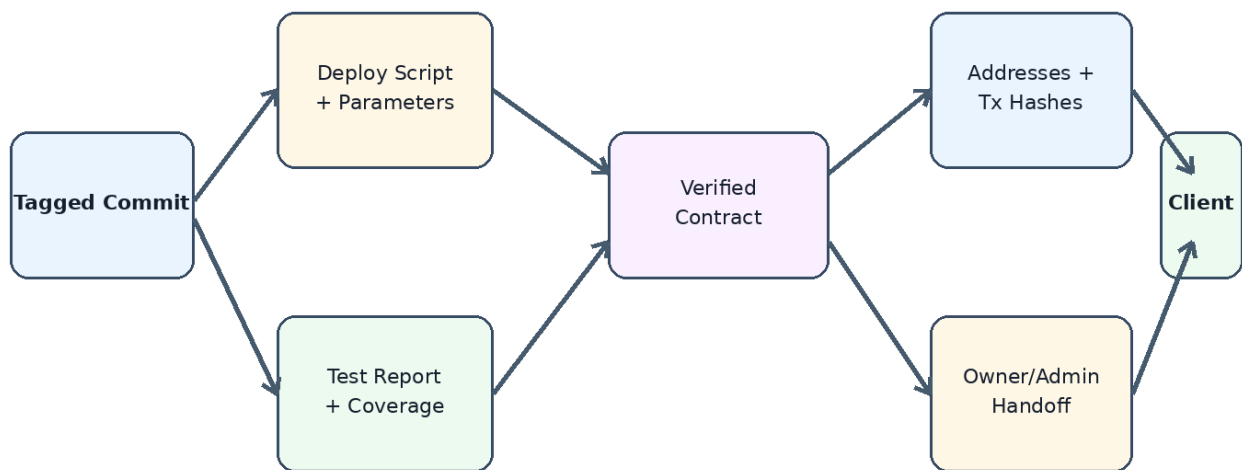
```
// deployments/sepolia.json
{
  "chainId": 11155111,
  "commit": "abc1234",
  "contract": "ClientStaking",
  "address": "0x...",
  "deploymentTx": "0x...",
  "blockNumber": 1234567,
  "constructor": {
    "stakingToken": "0x...",
    "rewardsToken": "0x...",
    "admin": "0x..."
  },
  "compiler": "0.8.24",
  "verified": true
}
```

Never guess at release time	Record instead
Which commit?	Git commit/tag
Which network?	chain ID + human name
Which inputs?	constructor/config arguments
Who deployed?	deployer address
What happened?	deployment tx + block
What was handed over?	role/admin transactions
Can source be inspected?	verification URL/status

22. Testnet Release Procedure

1. Freeze requirements for the release candidate.
2. Create/tag a release commit.
3. Run formatter, build, unit, fuzz and invariant tests.
4. Review privileged roles and constructor/deployment parameters.
5. Deploy to the target testnet with the scripted process.
6. Verify source code on the explorer.
7. Run smoke tests from at least two user wallets.
8. Fund a reward campaign with test tokens.
9. Test approve -> stake -> time/claim -> withdraw flow.
10. Test wrong-network UI and rejected/reverted transaction UX.
11. Run event indexer/backfill and compare against explorer logs.
12. Record every address, tx hash and test result.
13. Send acceptance checklist to client for sign-off.

Deployment and Handoff Evidence



23. Contract Verification and Smoke Tests

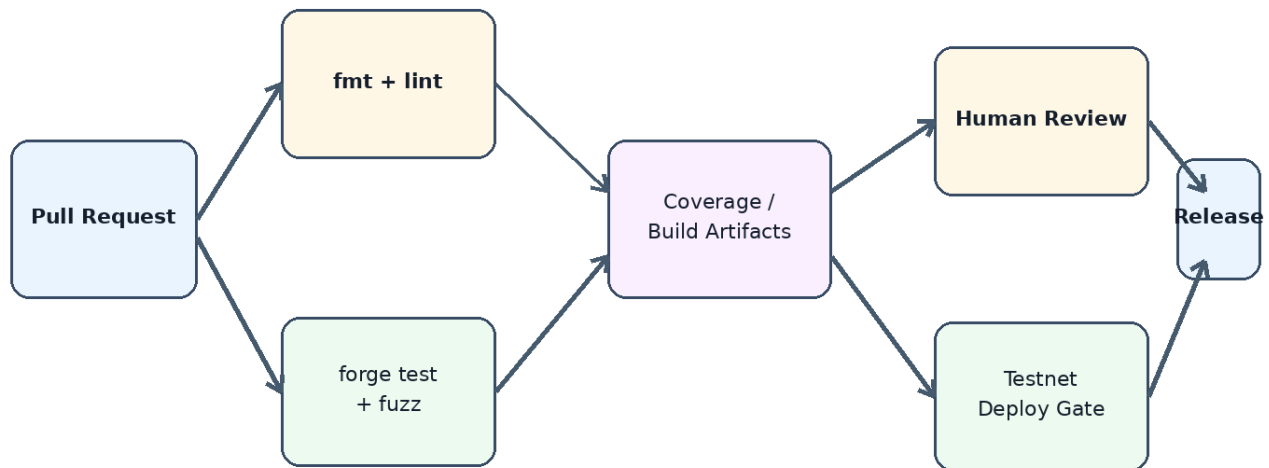
Verification is part of delivery evidence. A verified contract lets the client and third parties inspect the source associated with deployed bytecode.

Smoke test	Expected evidence
Read stakingToken()	Matches approved token address.
Read rewardsToken()	Matches approved reward token.
Read admin role	Client-designated admin is authorized.
Stake small amount	Success tx + Staked event + balance change.
Earned read	Returns expected non-negative value.
Claim	RewardPaid event and token balance change.
Withdraw	Principal returned and accounting reduced.
Unauthorized admin call	Reverts.
Pause policy	Matches specification.
Frontend	Uses exact deployed address for chain ID.

Do not skip tiny smoke tests: A perfectly tested contract can still be deployed with the wrong constructor address, wrong chain or wrong frontend configuration. Release QA checks the deployment, not just the source code.

24. CI: Automate the Boring Quality Gates

Repository CI and Release Gate



```

name: solidity-ci
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: foundry-rs/foundry-toolchain@v1
        with:
          version: stable
      - run: forge fmt --check
      - run: forge build
      - run: forge test -vvv
  
```

For private repositories and deployment workflows, keep sensitive values in appropriate secret stores and minimize workflow permissions. Do not print deployer secrets, RPC credentials or API tokens into logs.

Release policy: CI passing is necessary but not sufficient. Production deployment should still require explicit human review of the release commit, target chain, parameters, signer and ownership transfer.

25. Project Management: Milestones the Client Can Review

Milestone	Deliverable	Client review
M1 - Specification	Requirements, scope, architecture, acceptance criteria	Approve behavior and assumptions
M2 - Contract alpha	Solidity + unit tests	Review functions, roles, events
M3 - Security/testing	Fuzz/invariant tests + security notes	Review risks and limitations
M4 - dApp integration	Frontend wallet/read/write flow	Usability acceptance
M5 - Testnet release	Verified contracts + deployment record	Client test transactions
M6 - Handoff	Docs, admin transfer, final report	Final acceptance / support plan

25.1 Why milestones help

- The client sees progress before the end.
- Requirement mistakes are caught before expensive deployment.
- Payments can map to verifiable deliverables.
- Scope changes become visible instead of silently consuming time.
- Security decisions receive explicit client approval.

26. Estimation and Change Requests

Professional estimation is based on scope and uncertainty, not only lines of code.

Work item	Relative effort / uncertainty
Simple immutable staking core	Medium
Custom tokenomics / variable APY	High - economics clarification required
Upgradeable proxy	Higher security and testing burden
Frontend integration into existing clean repo	Medium
Frontend rescue in unknown legacy repo	High uncertainty
Indexer/API	Medium
Cross-chain staking	Very high - different trust model
Third-party audit remediation	Depends on findings

26.1 Change-request example

Original scope Single-token staking on one EVM chain with fixed campaign rewards.

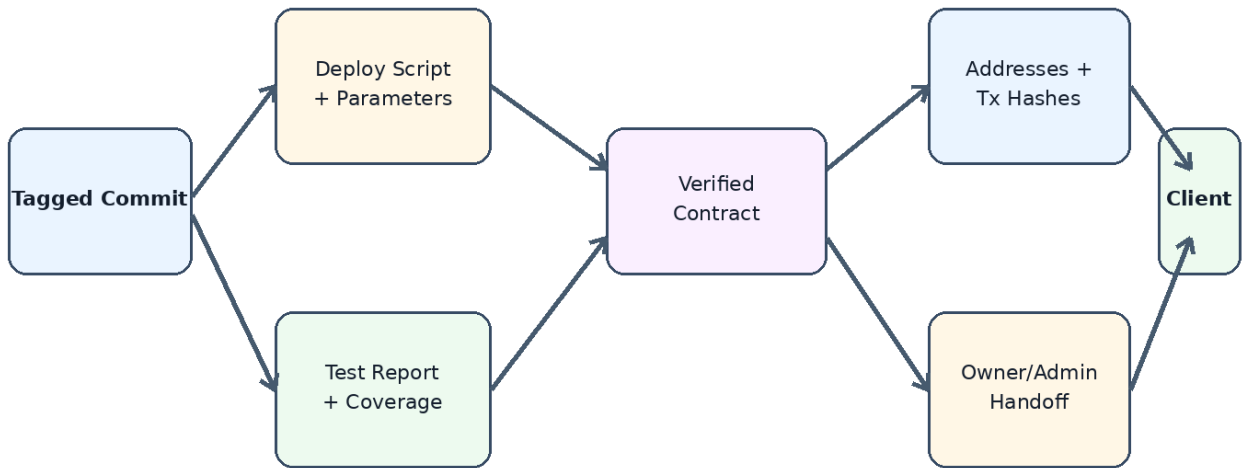
Client change Add NFT boost multipliers, referral rewards and Arbitrum deployment.

Correct response: document the new requirements, security implications, tests, schedule and price before implementation. Do not quietly absorb architecture-changing work into the original fixed scope.

27. Client Handoff Package

Artifact	What the client should receive
Source repository	Clean commit history / final release tag
README	Install, build, test, run, deploy
Requirements	Approved behavior + assumptions
Architecture	Components and trust boundaries
Security notes	Roles, limitations, pause/recovery policy
Test report	Commands + results + important cases
Deployment record	Chain, addresses, tx hashes, constructor inputs, commit
Verified contracts	Explorer verification evidence
Frontend env template	No secrets; exact required variables
Admin handoff	Role/ownership transfer tx evidence
Runbook	How to pause, fund rewards, diagnose common failures
Known issues	Explicit unresolved items / future work

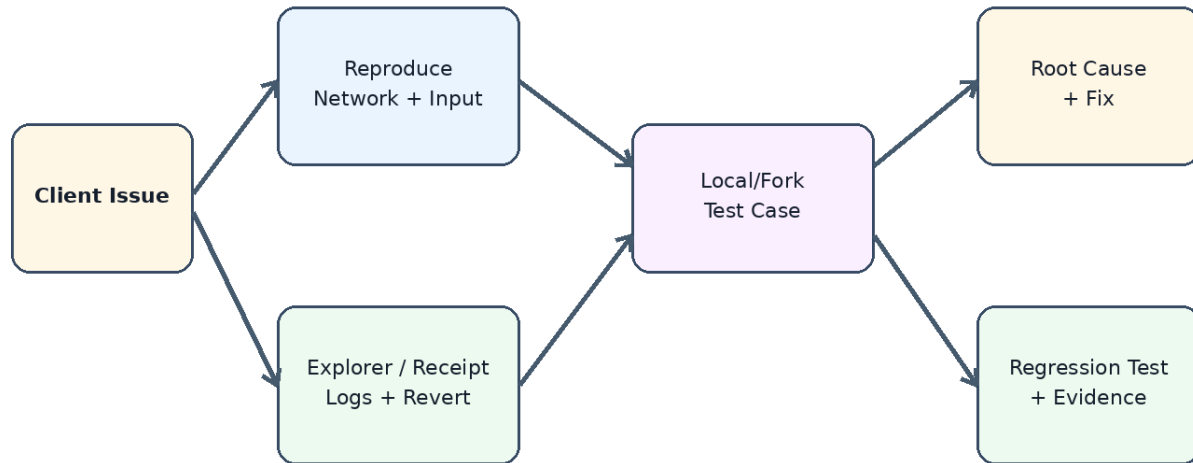
Deployment and Handoff Evidence



28. Real Client Debugging Scenario

Client report Users can approve the token, but stake() keeps reverting on the testnet. The UI only says "transaction failed."

Production Debugging Flow



1. Confirm chain ID and staking contract address used by the frontend.
2. Open the failed transaction or simulation result; inspect revert data.
3. Check allowance(owner, stakingAddress) and user token balance.
4. Confirm token address passed to the deployed staking contract.
5. Reproduce the exact caller/amount in a local or fork test.
6. Check pause state and amount validation.
7. If token is non-standard, inspect transferFrom behavior.
8. Write a regression test for the discovered cause.
9. Improve frontend error decoding so the next failure is actionable.
10. Send the client root cause, fix commit, test evidence and redeploy instructions.

29. Upwork / LinkedIn Job Translation

Job post wording	What you should translate it into
"Need staking smart contract"	Reward economics, custody model, roles, pause policy, token compatibility, tests.
"Need web3 frontend"	ABI, chain config, provider/signer, reads/writes, wallet states, transaction UX.
"Deploy contract to mainnet"	Release commit, parameter review, signer, gas, verification, ownership transfer, smoke tests.
"Fix contract bug"	Repro case, trace/revert reason, minimal fix, regression test, deployment impact.
"Make it secure"	Threat model and test scope; clarify whether independent audit is required.
"Need backend monitor"	Event schema, confirmations/reorgs, persistent cursor, dedupe, API/notification requirements.

29.1 Strong proposal structure

1. Restate the business goal in one sentence.
2. Identify 2-4 technical risks/questions from the post.
3. Propose milestones and concrete deliverables.
4. State the test/deployment evidence you will provide.
5. Ask only the questions needed to lock scope.
6. Avoid claiming "audited" or "100% secure" unless an actual audit exists.

30. Interview Questions + Day 18 Quiz

Question	Expected answer direction
Why write acceptance criteria before code?	To make scope testable and align client/developer definition of done.
What is a requirement traceability chain?	Need -> requirement -> implementation -> test/evidence.
Why is testnet deployment not enough?	Production parameters, signer, admin, dependencies and network conditions still need review.
Why use SafeERC20?	Safer interaction with ERC-20 implementations with differing return behavior.
Why separate roles?	Least privilege and safer operational administration.
What is a smoke test?	Small post-deployment check of the actual deployed system/configuration.
Why record commit and constructor args?	Reproducibility and deployment provenance.
What should a database do in this project?	Index/cache chain events; not redefine authoritative fund balances.
Why can fixed APY be a dangerous vague requirement?	It needs an exact formula, funding source and update policy.
How do you handle scope changes?	Document impact, re-estimate, obtain approval, then implement.

Quiz - answer without notes

1. List five discovery questions for a staking job.
2. Name three non-functional requirements.
3. What evidence proves a deployment is reproducible?
4. What should happen before a user balance changes in reward accounting?
5. Name three privileged actions that require explicit authorization.
6. Why keep deployment addresses in per-chain configuration?
7. What is the difference between a unit test and an acceptance test?
8. Name three items in a client handoff package.

31. Quiz Answer Key

1. Examples: token/network, reward formula/funding, roles, pause policy, lock rules, frontend/backend scope, audit requirement.
2. Examples: access control, reentrancy resistance, deterministic build, verification, documentation, performance/reliability expectations.
3. Release commit/tag, chain ID, address, deployment tx, constructor/config inputs, compiler/build settings and verification evidence.
4. Checkpoint global and user reward state using the previous balance before changing principal.
5. Examples: reward configuration/funding, pause/unpause, upgrade, recovery functions, role administration.
6. Because each network has different deployments and the app must choose the correct contract for the connected chain.
7. Unit tests verify individual code behavior; acceptance tests prove the delivered system meets client-agreed requirements.
8. Examples: repo, README, requirements, architecture, security notes, tests, deployment record, verified links, admin handoff and runbook.

Day 18 completion standard: You should be able to take a short client request and produce a one-page specification, architecture, milestone plan, test matrix and handoff checklist before you start coding.

32. Homework - Build the Portfolio Version

Turn today's simulated project into a portfolio repository. You do not need mainnet funds. Use local development and a public testnet when appropriate.

1. Create REQUIREMENTS.md with at least 10 functional/non-functional requirements.
2. Create ARCHITECTURE.md with the component and trust-boundary diagram.
3. Implement ClientStaking or a simplified equivalent using OpenZeppelin components.
4. Write at least 12 unit/negative tests.
5. Add two fuzz tests and at least one accounting invariant.
6. Build a minimal ethers.js v6 frontend with wallet connection, stake, earned, claim and withdraw.
7. Deploy to one EVM testnet and verify the contract.
8. Create deployments/<network>.json with release metadata.
9. Create HANDOFF.md with admin roles, reward funding procedure, pause policy and known limitations.
10. Write a one-page client completion report with links/evidence.

Portfolio README headline

Client Staking dApp - End-to-End Blockchain Delivery Demo
Solidity + OpenZeppelin + Foundry + ethers.js
Requirements -> architecture -> tests -> testnet -> verification -> handoff

33. Day 18 Freelancer Cheat Sheet

Before coding	Before deployment	Before handoff
Confirm token + chain	Freeze release commit	Verify contracts
Define reward economics	Run all tests	Record addresses/tx hashes
Define admin roles	Review parameters	Transfer/admin roles safely
Write scope/out-of-scope	Confirm signer + chain	Deliver README/runbook
Write acceptance criteria	Deploy testnet first	Deliver test/security evidence
Choose upgradeability policy	Smoke test actual deployment	List limitations/open issues
Define frontend/backend scope	Check wrong-chain config	Get client acceptance

Professional workflow to memorize

BRIEF

- > DISCOVERY
- > REQUIREMENTS
- > ACCEPTANCE CRITERIA
- > ARCHITECTURE
- > IMPLEMENTATION
- > SECURITY REVIEW
- > TESTS
- > TESTNET RELEASE
- > CLIENT ACCEPTANCE
- > PRODUCTION RELEASE
- > HANDOFF + SUPPORT

34. Primary Technical References & Day 19 Preview

Use official documentation as the source of truth when you build the portfolio project or deliver a client project:

- Foundry documentation - <https://getfoundry.sh/> - build, test, fuzz, debug, deploy and verify Solidity projects.
- Foundry best practices - <https://getfoundry.sh/best-practices> - project/test/script and security guidance.
- OpenZeppelin Contracts 5.x - <https://docs.openzeppelin.com/contracts/5.x> - reusable token, access-control and security components.
- OpenZeppelin Access Control - <https://docs.openzeppelin.com/contracts/5.x/access-control> - role-based authorization patterns.
- ethers.js v6 contracts/providers - <https://docs.ethers.org/v6/> - frontend/backend blockchain interaction.
- GitHub Actions secure use - <https://docs.github.com/en/actions/reference/security/secure-use> - workflow security practices.
- GitHub Actions secrets - <https://docs.github.com/actions/security-guides/using-secrets-in-github-actions> - secret storage for workflows.

Day 19 preview: Upwork + LinkedIn Blockchain Freelancing - profile positioning, portfolio packaging, job filtering, proposal structure, requirement questions, estimates, milestones, client communication, interview preparation and avoiding unprofitable/scam work.

End of Day 18 - You now have the delivery process that connects the technical skills from Days 1-17 to real paid blockchain work.